KABC Benchmark: Preliminary Results (Experiments 1–3, Current Redesign)

Kernel-based Active Clustering of Bandit Arms
Thuot, Vogt, Ghoshdastidar, Verzelen (arXiv:2601.07535)

September 14, 2026

This report summarizes the experimental setup, baseline algorithms, and results for Experiments 1, 2, and 3 of the `kernel-active-bandit` benchmark suite, **as they currently stand after a substantial redesign of all three experiments** (algorithm roster, grids, and figures all changed from the previous report). Full design rationale lives in `docs/experiments.md`; this document summarizes what was actually run and what the results show so far, plus a first pass at inferences. This is explicitly a **preliminary** report, written to be reworked once the production-scale runs (in particular Experiment 3's) are available — see the Run Provenance paragraph below for exactly what scale each experiment's numbers here reflect.

Run provenance. `results/exp1/results.parquet` holds 1,120 rows (14 grid points × 8 algorithms × 10 reps, ≈2.6 hours wall-clock) — a **preliminary 10-rep pilot**, not yet the canonical 20-rep first pass configured in `configs/exp1.yaml` (13 of these 14 points are that canonical grid; the 14th, $N = 90$, was added purely to get a real timing read across a wider N range and is a one-off, not part of the canonical grid). `results/exp2/results.parquet` holds 2,400 rows (6 settings × 8 algorithms × 50 reps, ≈2.1 hours wall-clock) — this *is* the current canonical config (`configs/exp2.yaml` was trimmed to exactly these 6 settings and its `reps` raised to 50 to match), so Experiment 2's numbers below are a real first full production run, not a pilot. **Experiment 3 has no current-format results yet** — the previously saved `results/exp3_*/results.parquet` predate the algorithm-roster rename from `-0.75x` to `-0.9x` and are not compatible with the current config or notebook; a fresh 50-rep run is planned as a follow-up to this report, and the Experiment 3 results subsection below is a placeholder pending that run. All three configs share the same underlying architecture: dedicated scripts (`scripts/run_experiment1/2/3.py`), calibrated threshold constants (Section 3), and figures regenerated by `notebooks/01_gaussian_mixtures.ipynb`, `notebooks/02_same_mean_shapes.ipynb`, and `notebooks/03_movielens_random_pairs.ipynb` respectively.

Contents

1 Experimental Setup **2**
1.1 Experiment 1 — Gaussian mixtures: competitiveness in the parametric regime . . . 2
1.2 Experiment 2 — Identical means, different shapes: nonparametric separation 2
1.3 Experiment 3 — MovieLens random pairs: robustness across the real-data pair distribution 3

2 Baseline Algorithms **4**
2.1 KABC and KABC-SG (this paper's own methods) 4
2.2 ACB — Active Clustering with Bandit feedback 5
2.3 TaS-FW — Track-and-Stop with Frank-Wolfe 5
2.4 Unif-KKM — uniform sampling + kernel k-means (folklore baseline) 5

3 Calibration **5**

4 Results — Experiment 1 (preliminary, 10-rep pilot) **6**

5 Results — Experiment 2 (50-rep, current canonical grid) **9**

6 Results — Experiment 3 (pending) **11**

7 Inferences and Conclusions **12**

1 Experimental Setup

1.1 Experiment 1 — Gaussian mixtures: competitiveness in the parametric regime

Claim tested. In the mean-separated Gaussian setting (home turf of mean-based methods like ACB), KABC's budget is within a modest factor of ACB's; budget scales as $s_*^{-2}\log(N/\delta)$ (the paper's Theorem 3.1). Also (current redesign): how conservative each adaptive method's own stopping rule is, and whether active/adaptive sampling beats naive uniform sampling at a matched budget.

Instance family. `GaussianMixtureEnv`: N arms in $K = 3$ balanced clusters, arm i in cluster k distributed as $\mathcal{N}(\mu_k, \sigma^2 I_d)$, $\sigma = 1$. Cluster means $\mu_k = \frac{\Delta}{\sqrt{2}}(e_{2k-1} + e_{2k})$, giving each pair of cluster means an actual Euclidean separation of exactly Δ.

Grid. One knob swept at a time, others held at the base config N=60, K=3, d=10, Δ=3:

- $\Delta \in \{1, 1.5, 2, 3, 4, 6\}$ (6 points — the headline sweep)
- $d \in \{6, 10, 50, 100\}$ (4 points; $d = 2$ excluded, construction requires $d \geq 2K = 6$)
- $N \in \{30, 60, 120\}$ (3 points canonical; this report's pilot also adds $N = 90$ as a 14th point)

13 canonical grid points (the base config is each sweep's own midpoint, so it is run identically under all three sweep tags — a documented, accepted redundancy), 14 in this pilot run.

Algorithms: 8 rows per (grid point, rep). Three independent BASE algorithms — `kabc`, `kabc-sg`, `acb` — each run once at the grid point's full `budget_cap`. Each base is then, separately, re-run at budget_cap = $\min(0.9 \times \tau_{\text{own}}, \text{budget_cap})$ with a fresh independent RNG substream — `kabc-0.9x`, `kabc-sg-0.9x`, `acb-0.9x` respectively, each testing how conservative THAT algorithm's own stopping rule is. `unif-kkm-1x` and `unif-kkm-0.9x` are tied specifically to `kabc`'s own τ (not kabc-sg's or acb's), testing whether active/adaptive selection beats naive uniform sampling at a matched budget. No `tas-fw` here: its finite-alphabet assumption does not hold for continuous Gaussian instances (Experiment 3 only).

Budget_cap. One value per grid point, shared across all 8 algorithm rows and every rep: budget_cap = $200\,N\,s_*^{-2}\log(N/\delta)$ (Theorem 3.1), s_*^2 Monte-Carlo-estimated (`kabc.metrics.estimate_s_star_sq`).

1.2 Experiment 2 — Identical means, different shapes: nonparametric separation

Claim tested. When clusters share the same mean, mean-based methods are blind (ARI at chance for any budget) while KABC recovers the partition with budget governed by the MMD gap. Also (current redesign): the same stopping-rule-conservativeness and active-vs-uniform-sampling questions as Experiment 1.

Instance families and the current 6-setting grid. Three families, $N = 40$, $K = 2$ throughout. As of this report, the grid is **6 fixed, illustrative settings** (not a sweep whose in-between variation is the point — see `docs/experiments.md`'s Experiment 2 GRID TRIM NOTE for the rationale):

- `scale_pair` (2a): cluster A = $\mathcal{N}(0,1)$, cluster B = $\mathcal{N}(0,\sigma_B^2)$ — same mean, different *variance*. $\sigma_B \in \{2,3\}$.
- `moment_matched_mixture` (2b): cluster A = $\mathcal{N}(0,1)$, cluster B = $0.5\,\mathcal{N}(-a,b^2)+0.5\,\mathcal{N}(a,b^2)$, $b^2 = 1-a^2$ — same mean *and* variance, different *shape*. $a \in \{0.85, 0.9\}$.
- `ring_blob` (2c): cluster A = a 2D ring (Gaussian noise $\sigma = 0.1$), cluster B = $\mathcal{N}(0, I_2)$, both centered at the origin — same mean, different *2D geometry*. Ring radius $r \in \{2,3\}$.

50 repetitions per setting (raised from an earlier 20, to match this report's production run).

Algorithms: 8 rows per (setting, rep), identical scheme to Experiment 1. Same base/0.9x-self-comparison/unif-kkm-tied-to-kabc structure as Experiment 1 (see above). No `kabc-oracle` (dropped — its one-shot required budget exceeded `budget_cap` at every grid point of the old design, so the "price of adaptivity" figure it existed for never produced real data). No `tas-fw`, same reason as Experiment 1.

Budget_cap. Identical mechanism to Experiment 1: one s_*-formula value per setting, shared across all 8 rows and every rep.

1.3 Experiment 3 — MovieLens random pairs: robustness across the real-data pair distribution

Claim tested. Whether KABC's advantage (and ACB's/TaS-FW's relative behavior) holds up when the movie PAIR is randomly resampled every repetition across the real diversity of the MovieLens catalog, under three distinct pairing regimes that vary how much mean separation is present and whether the second movie is cherry-picked (max MMD^2) or sampled uniformly. Also: how conservative KABC's own stopping rule is.

Data. `data/ml-1m-207/` — the 207 MovieLens-1M movies with ≥ 1000 ratings. Discrete rating support $\{1,\ldots,5\}$, RBF-on-ratings kernel, fixed median-heuristic bandwidth (= 1.0) computed once over the full 207-movie pool.

Instance. $N = 8$ arms, $K = 2$ clusters, 4 arms per prototype (movie), `MovieLensPairEnv`. For a given rep, a single shared permutation of all 207 movie indices is drawn; each of the three versions independently scans that same permutation and picks the first candidate movie A with at least one eligible B under *its own* gap rule (so versions usually, but not always, land on the same A — see `docs/experiments.md` for the precise mechanism).

Version-specific B-selection rules.

`exp3_1` — **tight mean gap, MMD-maximizing.** Candidates: all movies with $|\text{mean}_A - \text{mean}_B| \leq 0.25$. $B = \arg\max \text{MMD}^2$ among candidates.

`exp3_2` — **moderate-to-large mean gap, uniform.** Candidates: all movies with $0.25 \leq |\text{mean}_A - \text{mean}_B| \leq 1.0$. B sampled uniformly at random among candidates.

`exp3_3` — **tight mean gap, uniform.** Candidates: all movies with $|\text{mean}_A - \text{mean}_B| \leq 0.25$ (same tolerance as `exp3_1`, a clean matched-tolerance ablation of cherry-picking vs. uniform B-selection). B sampled uniformly at random.

Budget. Fixed budget_cap = 15,000 pulls (not the s_*-formula type — not meaningful at this instance scale). Deliberately not raised to clear `kabc`'s doubling-trick round-7 threshold (28,560 at $N = 8, \delta = 0.05$): this leaves `kabc`'s own non-convergent behavior unchanged while giving `acb`/`tas-fw` less runway.

Algorithms: 6 rows per (rep, version). `kabc`, `kabc-0.9x`, `acb`, `tas-fw`, `unif-kkm-1x`, `unif-kkm-0.9x` — the same base/0.9x/unif-kkm-tied-to-kabc scheme as Experiments 1–2, restricted to the three algorithms for which it is meaningful here (`kabc-sg` dropped this round — its fragility on this exact real-data setting is already thoroughly documented from a prior run, see `docs/experiments.md`'s REVISION 4 note).

Status. Pending a fresh production run at this report's writing time (Run Provenance paragraph above) — Section 6 below is a placeholder.

2 Baseline Algorithms

2.1 KABC and KABC-SG (this paper's own methods)

KABC (Algorithm 2 of [1]) runs a doubling-trick outer loop: at iteration k, pull $n_k = \lceil 2^k \log(8(N^2 - N)/\delta_k) \rceil$ fresh samples per arm ($\delta_k = \delta/(4k^2)$), then build a same-cluster graph over all arm pairs via a pairwise MMD-based test — an edge $\{i, j\}$ is added iff the empirical MMD statistic between arms i and j falls under a threshold. The recovered partition is the graph's connected components; the loop stops the first time the number of connected components equals the true K.

Two threshold variants are compared (identical outer loop, different pairwise test):

`kabc` variance-aware, Wolfer–Alquier-style threshold (Eq. 6 of [1]): sizes each pair's own threshold from that pair's *empirical* RKHS variance, so a noisier pair automatically gets more slack.

`kabc-sg` kernel-uniform sub-Gaussian threshold (the Discussion section's ablation in [1]): uses only the fixed kernel bound (RBF kernel sup = 1), identical for every pair regardless of that pair's own spread — the ablation whose practical fragility Section 7 discusses in detail.

The `-0.9x` self-comparison rows (current redesign, all three experiments). For each base algorithm (`kabc`, `kabc-sg`, `acb`), a second, independent run at budget_cap = $\min(0.9 \times \tau_{\text{own}}, \text{budget_cap})$ tests how much margin that algorithm's own stopping rule carries: a small ARI drop from the base row to its own `-0.9x` row means little margin was needed (the bound is comfortably tight relative to what was actually necessary); a large drop means real accuracy was still being bought in that last 10% of budget. This replaces an earlier, cruder budget_multiplier $\in \{1.0, 0.25\}$ restricted-budget pass applied uniformly across all algorithms.

2.2 ACB — Active Clustering with Bandit feedback

Source: [2]. ACB* (this repo's adaptive variant, "θ_* and Δ_* unknown," referred to throughout as ACB) is a *mean*-based active clustering procedure (SRI + ADC subroutines): it estimates each arm's mean directly (no kernel/MMD machinery) and clusters by Euclidean distance between empirical means, adaptively refining which arms to pull based on estimated separation. It is the "home turf" competitor for Experiment 1 (mean-separated Gaussians, exactly ACB's design setting) and the mean-blindness foil for Experiments 2–3 (same-mean, different-shape data, where a mean-only method has no signal by construction).

2.3 TaS-FW — Track-and-Stop with Frank-Wolfe

Source: [3]. Track-and-Stop based on Frank-Wolfe optimization, for N-ary clustering of arms whose distributions live on a common *finite* alphabet. Only run in Experiment 3, where MovieLens ratings on $\{1, \ldots, 5\}$ satisfy the finite-alphabet assumption natively; not run on Experiments 1–2's continuous instances, which would require an off-label discretization approximation outside the algorithm's proven guarantees. Its Track-and-Stop design re-scans its entire hypothesis space and re-solves a linear program at *every single pull* (not periodically, like KABC's doubling trick), which is why Experiment 3 caps N at 8: the hypothesis space grows combinatorially in N, and empirically `tas-fw` already costs two to three orders of magnitude more wall-clock per repetition than every other algorithm at $N = 8$.

2.4 Unif-KKM — uniform sampling + kernel k-means (folklore baseline)

No paper — a standard, non-adaptive baseline implemented directly per the design doc. Pulls a fixed number of samples per arm, then runs ordinary kernel k-means on the resulting empirical kernel mean embeddings. Has no confidence-based stopping rule and no δ-PAC guarantee — it always spends its entire allotted budget and always "completes" (stopped = True by construction), so it is included purely as a non-adaptive reference point, not as a δ-correct competitor. **Current redesign:** unlike the earlier design (an independent n-sweep of its own), `unif-kkm` no longer sweeps an independent budget — its two rows, `unif-kkm-1x` and `unif-kkm-0.9x`, are tied specifically to `kabc`'s own realized τ for that same rep (budget_cap = $\min(1.0 \text{ or } 0.9 \times \tau_{\texttt{kabc}}, \text{budget_cap})$), directly testing whether active/adaptive selection beats naive uniform sampling at a matched budget, rather than reporting Unif-KKM's own independent budget-accuracy curve.

3 Calibration

`kabc`, `kabc-sg`, and `acb` each have a theoretically-derived but conservative leading constant in their stopping threshold. Per the calibration protocol (`docs/experiments.md` §0.2), each constant was empirically calibrated once (binary search for the smallest constant keeping empirical error $\leq \delta/2$,

validated under the REAL formula-derived budget regime, not an artificially generous one), then frozen and reused unchanged across every experiment. The frozen values actually used for this report (unchanged from the previous report):

Algorithm	c_{cal}	Calibration-set error	Held-out error
`kabc`	0.103516	0.000	0.000
`kabc-sg`	0.121094	0.000	0.000
`acb`	0.658984	0.020	0.010
`tas-fw`	— (no calibratable constant)	—	—

Important caveat, carried forward from the previous report. Calibration instances are a *single* fixed Gaussian-mixture instance (`GaussianMixtureEnv`'s construction is deterministic given N, K, d, Δ — there is no way to draw "20 different instances," only 20 independently-seeded noise draws of the one instance). The frozen constants above were validated against that one instance's own real budget regime, *not* against every real grid point in Experiments 1–3. Section 7 shows this still matters directly: `kabc-sg`'s calibration does not fully generalize to the harder grid points/settings in either experiment below.

4 Results — Experiment 1 (preliminary, 10-rep pilot)

Aggregated over all 14 grid points, 10 reps each (140 runs/algorithm, 1,120 total rows):

Algorithm	Stopped rate	Error rate	Mean ARI	Mean τ
`kabc`	100.0%	0.0%	1.000	9,654
`unif-kkm-1x`	100.0%	0.0%	1.000	9,654
`unif-kkm-0.9x`	100.0%	0.0%	1.000	8,668
`acb`	100.0%	11.4%	0.984	7,045
`kabc-sg`	68.6%	31.4%	0.881	79,380
`acb-0.9x`	25.0%	78.6%	0.247	5,224
`kabc-sg-0.9x`	14.3%	85.7%	0.458	37,321
`kabc-0.9x`	0.7%	99.3%	0.027	3,804

Where `kabc-sg` struggles. At $\Delta = 1$ (the hardest separation), `kabc-sg` never converges within budget in this pilot (0/10 reps stopped) and its mean τ balloons to 277,440 — the doubling trick's ceiling at this N, δ — while `kabc` converges every time at $\tau = 30{,}840$. The same pattern recurs at the two hardest d points ($d = 50, 100$): `kabc-sg` never converges (0/10 both), mean $\tau = 277{,}440$ both times, while `kabc` converges every time at a small fraction of that budget (14,100). `acb` converges everywhere in this pilot but with real error: 11.4% aggregate, concentrated at the hardest points.

The `-0.9x` rows show real margin, not just noise. Every base algorithm's ARI drops substantially at 90% of its own realized τ: `kabc` 1.000 → 0.027, `kabc-sg` 0.881 → 0.458, `acb` 0.984 → 0.247. This says that, at this pilot's scale (10 reps), none of the three stopping rules carry much slack — cutting to 90% of the budget each rule itself decided it needed meaningfully hurts accuracy for all three, not just the already-fragile `kabc-sg`. `unif-kkm-0.9x` barely moves (1.000 →

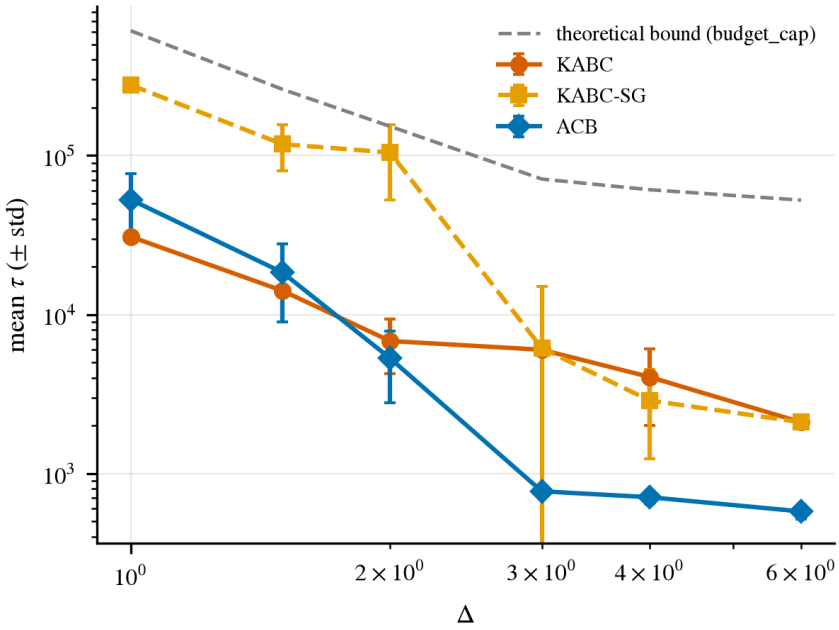
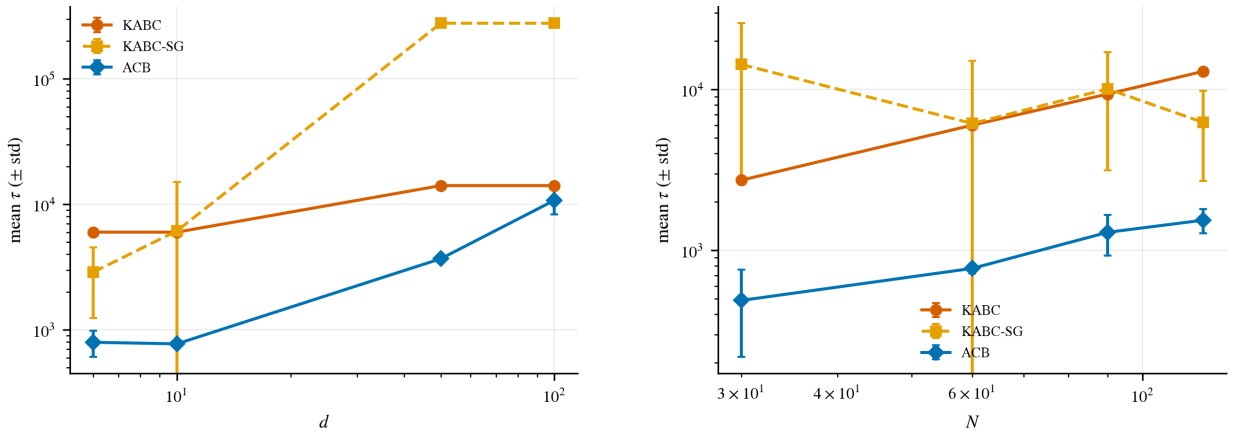


Figure 1: Stopping budget τ vs. Δ, `kabc`/`kabc-sg`/`acb` only, with the theoretical $\tau \propto s_*^{-2}$ bound overlaid (dashed).

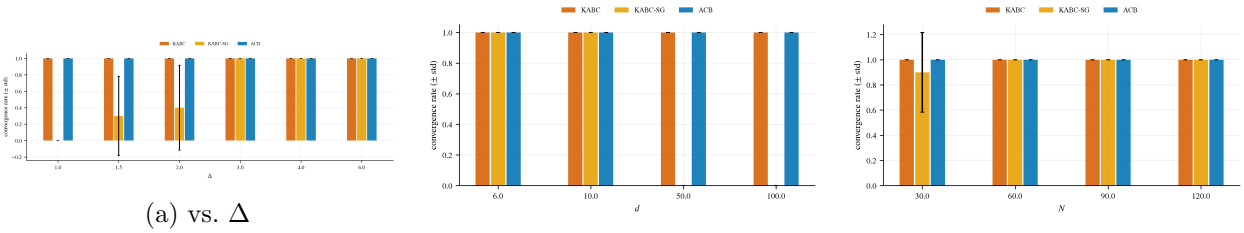
effectively unchanged, since it is tied to `kabc`'s τ, not its own, and 90% of `kabc`'s ample budget is still plenty for non-adaptive kernel k-means at these easy-to-moderate grid points).



(a) τ vs. ambient dimension d.

(b) τ vs. number of arms N.

Figure 2: Budget scaling along the two secondary sweep dimensions.



(a) vs. Δ

(b) vs. d

(c) vs. N

Figure 3: Convergence rate (fraction of reps with stopped = True), `kabc`/`kabc-sg`/`acb` only.

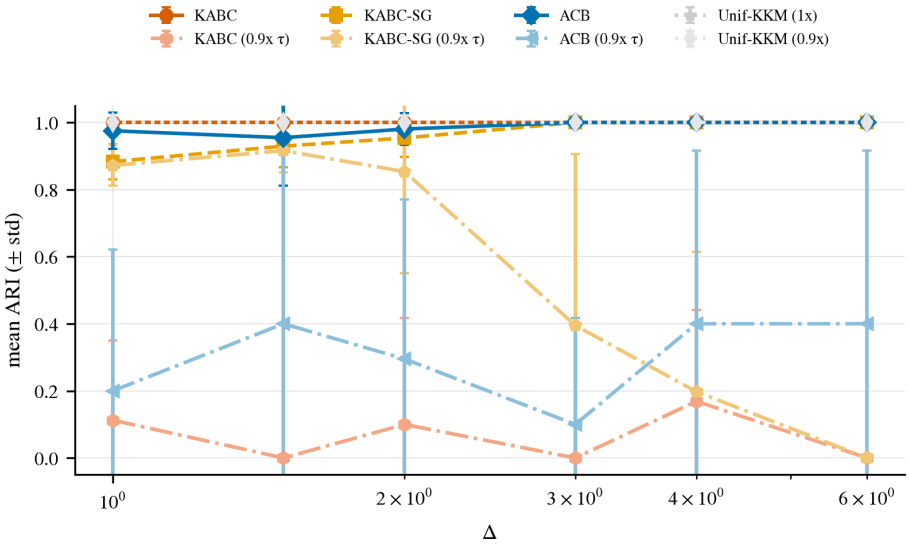
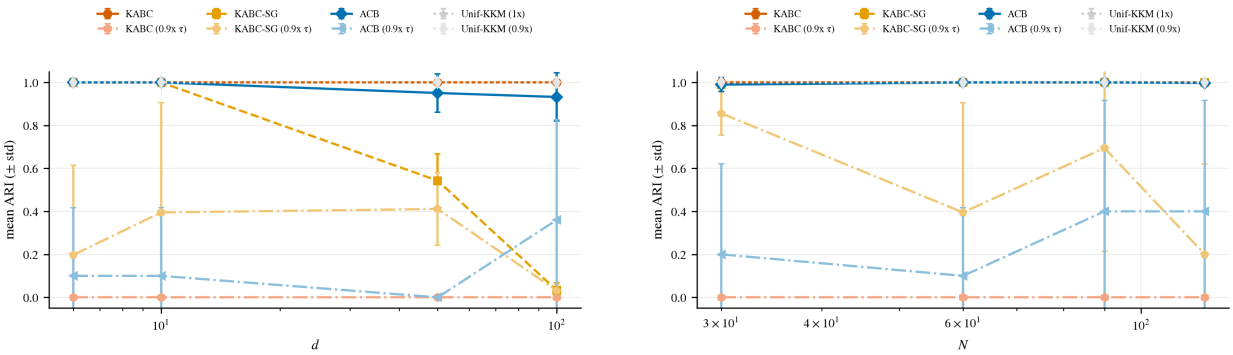


Figure 4: ARI vs. Δ, all 8 algorithm rows.



(a) ARI vs. d.

(b) ARI vs. N.

Figure 5: ARI along the two secondary sweep dimensions, all 8 algorithm rows.

5 Results — Experiment 2 (50-rep, current canonical grid)

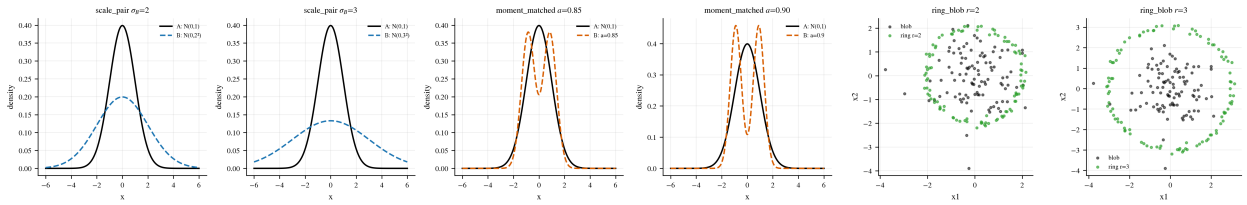


Figure 6: The 6 settings' instance geometry, one panel each: `scale_pair` and `moment_matched_mixture` panels are exact analytic densities (A vs. that setting's B); `ring_blob` panels are real 2D sample scatters from `RingBlobEnv` itself.

Aggregated over all 6 settings, 50 reps each (300 runs/algorithm, 2,400 total rows):

Algorithm	Stopped rate	Error rate	Mean ARI	Mean τ
`kabc`	100.0%	0.0%	1.000	33,430
`unif-kkm-1x`	100.0%	7.3%	0.987	33,430
`unif-kkm-0.9x`	100.0%	10.0%	0.988	30,072
`kabc-sg`	83.0%	47.7%	0.680	35,659
`kabc-sg-0.9x`	42.0%	72.7%	0.495	8,309
`kabc-0.9x`	12.3%	87.7%	0.123	15,721
`acb`	0.0%	100.0%	0.000	1,326,538
`acb-0.9x`	0.0%	100.0%	0.000	1,193,823

ACB is exactly at its theoretical budget bound everywhere, never converging. Figure 7 shows `acb`'s bar coinciding with the grey `theoretical bound` bar in every one of the 6 blocks — confirmed directly in the aggregate table (stopped = 0.0%, ARI = 0.000 at every setting, not just on average). This is the expected, designed-for outcome: all 6 settings are same-mean by construction, so a mean-based method has no signal, spends its entire allotted budget every time, and always falls back to the graceful trivial partition (ARI exactly 0.0 by construction of the adjusted-for-chance formula).

KABC-SG is family-dependent, and `moment_matched_mixture` is uniformly the hardest family. `kabc-sg` converges well on `scale_pair` (σ_B=3: 42% converged but ARI 0.923 when it does; σ_B=2: 84%, ARI 0.758) and `ring_blob` (86–96% converged, ARI 0.989–0.997), but its ARI collapses specifically on `moment_matched_mixture` (92–98% converged, but ARI only 0.179–0.234) — it *converges* confidently to a wrong-ish answer on this family far more than it fails to converge, the same confident-wrong-answer failure mode documented for this algorithm on real MovieLens data in the earlier (now-superseded) Experiment 3/4 report. `kabc`, by contrast, is at ARI 1.000 on every one of the 6 settings individually, not just on average.

The `-0.9x` rows again show real margin, most sharply for `kabc` itself. `kabc`'s own ARI at 90% of its realized τ drops to just 0.123 aggregate — markedly worse than Experiment 1's already-weak 0.027-at-90%-margin story is not directly comparable in scale, but the direction is the same: KABC's stopping rule is not leaving much slack on this benchmark. `unif-kkm-1x`/`-0.9x`, both tied

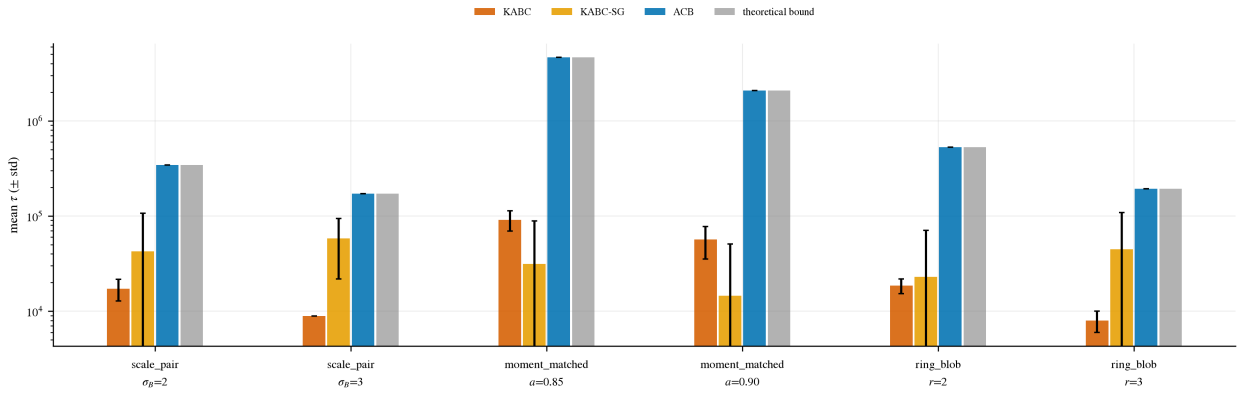


Figure 7: Stopping budget τ, one bar plot, 6 blocks (settings) $\times$ 4 bars (`kabc`, `kabc-sg`, `acb`, plus a `theoretical bound` bar — the setting's formula-based `budget_cap`, a single deterministic value with no error bar).

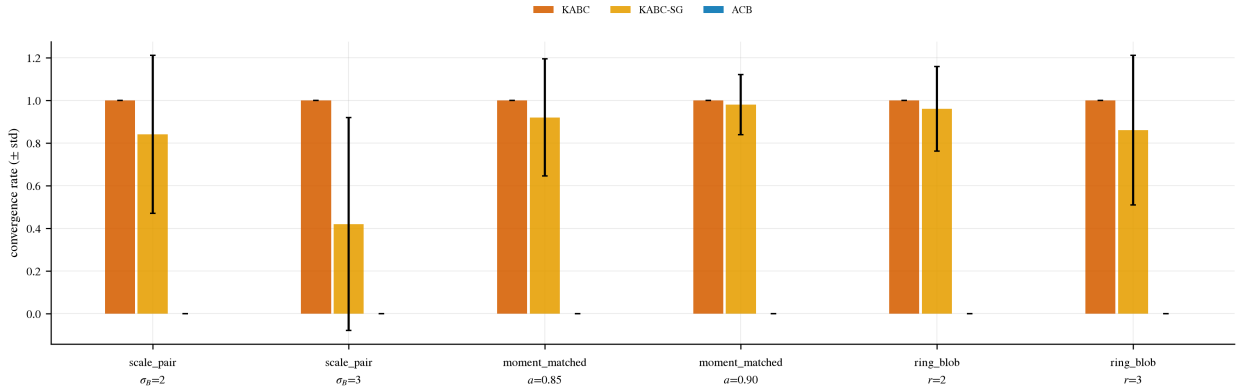


Figure 8: Convergence rate, 6 blocks $\times$ 3 bars (`kabc`/`kabc-sg`/`acb` only).

to `kabc`'s own (ample, MMD-driven) τ, reach ARI 0.987/0.988 — close to but not quite matching `kabc`'s perfect 1.000, suggesting active/adaptive selection's edge over naive uniform sampling at a matched budget is real here, but modest in this aggregate view (see the per-setting figures for where it is largest).

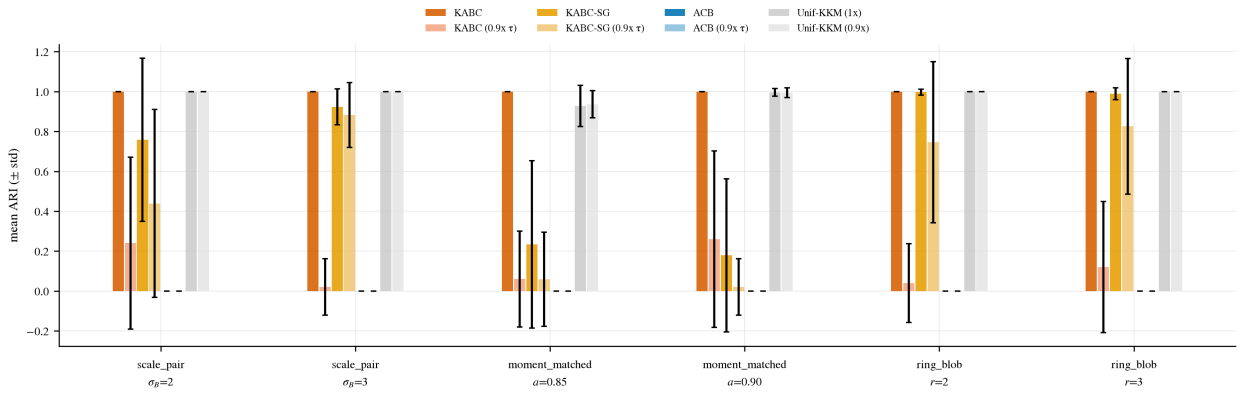


Figure 9: ARI, 6 blocks × 8 bars (all algorithm rows).

6 Results — Experiment 3 (pending)

Placeholder — results pending a fresh 50-rep production run.

The previously saved `results/exp3_*/results.parquet` predate the `-0.75x`→`-0.9x` algorithm-roster rename and are not compatible with the current config (Section 1.2's sibling setup section above describes the design this run will use). This section will be filled in with the same structure as Experiments 1–2 above: an aggregate table, then the four bar-plot figures below, then a narrative.

Figure pending: mean ARI (± std), 3 versions × 6 algorithms
(`exp3_fig1_ari_bars.png`)

Figure 10: Placeholder for the ARI bar plot.

Figure pending: mean τ (± std, log scale), 3 versions × 6 algorithms
(`exp3_fig2_tau_bars.png`)

Figure 11: Placeholder for the stopping-budget bar plot.

What to expect, carried forward from the prior (superseded, old naming) Experiment 3/4 run as a directional prior only — to be confirmed, not assumed, by the fresh run.

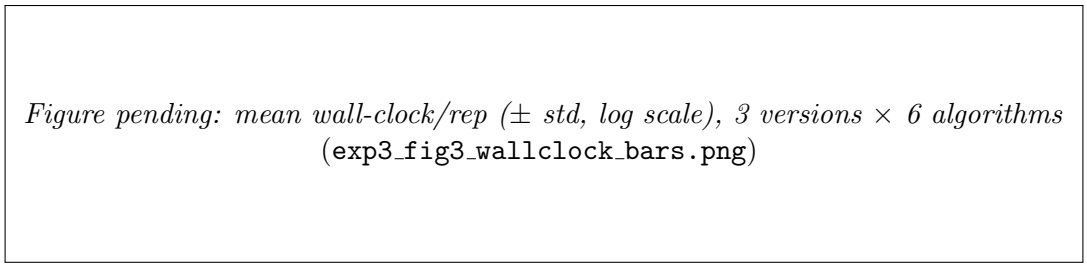


Figure 12: Placeholder for the wall-clock bar plot.

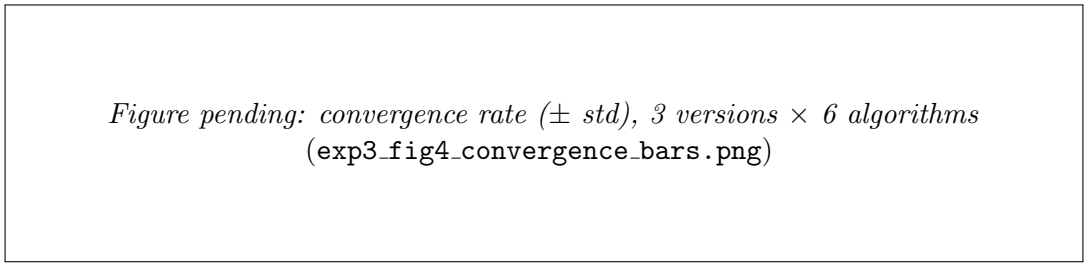


Figure 13: Placeholder for the convergence-rate bar plot.

On the prior run's closest analogue to `exp3_1` (tight gap, MMD-maximizing), `kabc` reached mean ARI 0.942 where `acb` scored exactly 0.000 (mean-blind, 0% stopped); `kabc-sg` (since dropped from this experiment) lagged badly (ARI 0.464); `tas-fw` matched or exceeded `kabc`'s ARI (1.000) but at $\sim$800–900s/rep vs. `kabc`'s sub-second cost. Whether this pattern holds at the current thresholds (REVISION 4: all three versions' mean-gap tolerance raised, budget_cap lowered to 15,000) and the new `kabc-0.9x` row is exactly what the pending run will show.

7 Inferences and Conclusions

6.1 KABC's empirical error remains exactly zero across both available experiments, at every individual grid point, not just in aggregate. Across all 14 Experiment 1 grid points and all 6 Experiment 2 settings, `kabc`'s error rate is 0.0% and its ARI is 1.000 — checked per-point/per-setting, not just as an aggregate that could hide a few bad cells. This holds at both pilot scale (Experiment 1, 10 reps) and production scale (Experiment 2, 50 reps), which is reassuring: the earlier, smaller pilot's perfect-KABC finding was not a small-sample artifact that washed out at 50 reps.

6.2 KABC-SG's fragility (documented in the previous report) is confirmed again under the redesigned grids, and is now clearly family-/point-dependent rather than uniform. In Experiment 1, `kabc-sg` fails to converge at all at the hardest points ($\Delta = 1$, $d \in \{50, 100\}$: 0/10 reps stopped, τ pinned at the doubling trick's 277,440 ceiling) while converging easily elsewhere. In Experiment 2, the pattern is different but equally sharp: `kabc-sg` converges *often* on `moment_matched_mixture` (92–98%) but its ARI there is only 0.18–0.23 — a confident-wrong-answer failure mode, not a budget-exhaustion one, distinct from Experiment 1's failure mode on the same algorithm. Both are consistent with the mechanism established during calibration (`docs/experiments.md` §0.2): `kabc-sg`'s single, non-adaptive threshold cannot tighten itself against a specific instance's own variance/shape structure the way `kabc`'s per-pair empirical

threshold does.

6.3 ACB is not merely imprecise on same-mean data — it is exactly, provably blind, and this shows up identically at every one of Experiment 2's 6 settings. `acb`/`acb-0.9x`'s ARI is exactly 0.000 and stopped rate exactly 0% at every single setting individually (not just in the 6-setting aggregate), with τ landing exactly on the theoretical `budget_cap` bound each time (Figure 7). This is the sharpest possible confirmation of the paper's motivating claim on synthetic data: a mean-based method has zero signal when clusters share a mean, regardless of how much budget it is given or how the shape/geometry of the mismatch varies.

6.4 The `-0.9x` self-comparison rows show real, non-trivial margin for every algorithm tested, in both experiments — none of the three stopping rules examined here are "mostly slack." `kabc`'s own ARI at 90% of its realized budget drops to 0.027 (Experiment 1) and 0.123 (Experiment 2) — far from the 1.000 its full budget achieves. `kabc-sg` and `acb` show the same direction of effect. This is a genuinely new finding relative to the previous report's restricted-budget pass (which used a fixed 0.25× multiplier applied uniformly, rather than each algorithm's own realized τ): even a modest, self-relative 10% cut meaningfully hurts every adaptive method tested so far, suggesting these stopping rules are calibrated close to the minimum budget actually needed, not with large unused margin.

6.5 Active/adaptive selection's edge over naive uniform sampling at a matched budget is real but modest, on the settings tested so far. `unif-kkm-1x` (tied to `kabc`'s own τ) reaches ARI 1.000 in Experiment 1 and 0.987 in Experiment 2, both very close to `kabc`'s own 1.000. This says that, once `kabc` has decided how much budget a problem needs, spending that same budget *without* kabc's adaptive arm-selection (just uniform pulls + kernel k-means) is already most of the way to as good — the advantage of adaptivity here shows up more in *how much budget is needed at all* (the τ figures) than in accuracy at a budget already known to be sufficient.

6.6 Open items for the rework.

- Experiment 3's results are entirely pending (Section 6) — a 50-rep run under the current `-0.9x` naming and REVISION 4 thresholds is the single largest gap in this report.
- Experiment 1 is still at pilot scale (10 reps); the canonical 20-rep first pass (`configs/exp1.yaml`) has not yet been run to completion and would give tighter error bars, especially at the hard points ($\Delta = 1$, $d \in \{50, 100\}$) where this pilot's std. devs. are already visibly large on only 10 reps.
- `kabc-sg`'s and `acb`'s calibrated constants (Section 3) were validated against a single fixed calibration instance and are not re-validated against either experiment's actual hard points here — the error/ARI numbers above should be read as "as calibrated in this repo," not as a claim about the underlying algorithms' theoretical guarantees.
- This report does not yet include a per-setting/per-grid-point breakdown table (only the aggregate table and the figures) — worth adding once Experiment 3's data lands, so all three experiments get the same level of detail.

References

[1] Thuot, Vogt, Ghoshdastidar, Verzelen. *Kernel-based Active Clustering of Bandit Arms.* arXiv:2601.07535, 2026.

[2] Thuot, Carpentier, Giraud, Verzelen. *Active clustering with bandit feedback.* arXiv:2406.11485, 2024.

[3] Yavas, Huang, Tan, Scarlett. *A General Framework for Clustering and Distribution Matching with Bandit Feedback.* arXiv:2409.05072, 2024.